

NUREEN AMINI BINTI FAIRUZ

B032420127

BITP2223 SOFTWARE REQUIREMENT AND DESIGN

Lab Task 9 – Set 1

Answer each question below. Your efforts to answer using your own sentences that are concise, neat, complete and structured will give you extra marks.

Question 1 (Total: 5 Marks)

- a) Explain the difference between cohesion and coupling in software design. (2 Marks)

Cohesion is a measure of how well the elements within a component fit together. A cohesive component performs a single logical function or responsibility in a highly cohesive way

Coupling is a measure of the strength of the inter-connections between system components. Low (loose) coupling means that they rely less on one another and that a change in one component will have a minimal impact on other components.

- b) Why is high cohesion and low coupling considered a good design practice? (2 Marks)

High cohesion is good because similar functionality tends to be kept in one component and making the system more understandable, maintainable and modifiable.

Low coupling is good since parts are more independent. Even though there are changes made to one component, the other components are not affected because it has improved the flexibility, reusability and maintainability.

- c) Give one simple example to support your answer. (1 Mark)

For example, A Student Management Module which only performs student-related tasks (Add, Update, Delete Student Records) has high Cohesion. Low coupling occurs when it communicates with the Payment Module only by method parameters or message passing and does not share variables directly.

Question 2 (Total: 5 Marks)

Software design is often described as a “blueprint” for software development. In your own words, explain what software design means and why it is important before the implementation (coding) phase begins.

Software design is a detailed plan or blueprint of a software system before it is implemented. In this phase, software engineers transform the system needs into a logical program that outlines the system's design, function, structure, and data management. The relevance of software design is that it will give guidance to software developers during the coding process and will give the software system a direction that will satisfy the customer. Besides, software design facilitates communication within the team and simplifies the future testing, maintenance, and improvement of the software.

Question 3 (Total: 10 Marks)

Software complexity metrics are essential tools used by engineers to evaluate design quality and predict maintenance effort during the system design phase.

- a) Distinguish between intra-modular complexity and inter-modular complexity by defining each term and providing one unique structural example or metric used to measure it. (4 Marks)

Aspect	Intra-Modular Complexity	Inter-Modular Complexity
Definition	Measures the complexity within a single module or component.	Measures the complexity of the relationships and interactions between different modules.
Example Metric	Depth of nesting, number of if-statements	Coupling

- b) Suppose a software design contains a module with a very high depth of conditional nesting (e.g., five levels of nested if-else blocks). Explain how this specific intra-modular metric value impacts software maintenance effort and suggest a design strategy to mitigate this complexity. (6 Marks)

The intra-modular complexity is high in the module having five levels of nested if-else statement. As a result, the code becomes hard to read and understand since the developers must follow numerous conditions to understand the program. This makes it longer to discover and correct mistakes, change the software, and test software. Thus, the maintenance requirement increases. With software complexity measures, the greater the complexity the more effort will be needed and the more difficult it will be to maintain the design. This can be simplified by dividing this nesting of if-else statements into smaller functions or methods. This will help to keep the code simple, easy to understand and easy to maintain in the future.